

Metalibros v2

Autor: Mauricio S. Matias Conde

Resumen Este proyecto es la segunda versión de Metalibros (trabajo #1), con mejoras en la navegación y el contenido visual, manteniendo la consistencia con la primera versión y principalmente empleando el conocimiento adquirido (desarrollo web cliente-servidor) en el primer módulo de la maestría.

Tabla de contenidos

Detalles Técnicos	2
Tecnologías Utilizadas	2
Endpoints y Rutas	3
Requisitos	5
Instalación	5
Configuración	5
Base de datos	6
Ejecución	6
Build de producción	6

Enlace Repositorio Público: <https://github.com/cr0wg4n/metalibros-v2>

Enlace Pull Request Realizados: <https://github.com/cr0wg4n/metalibros-v2/pulls>

Enlace Commits Realizados: <https://github.com/cr0wg4n/metalibros-v2/commits/main>

Metalibros, es una plataforma para la gestión y venta de libros, que permite a los usuarios publicar, editar y administrar sus libros, así como llevar un control de las ventas y el stock.

Esta segunda versión es la evolución del primer proyecto dentro del 1er módulo de la maestría, se empleó dicha primera versión como inspiración en temas de navegación y contenido visual, obteniendo así una base sólida y una apariencia muy similar, manteniendo la consistencia deseada.

Esta nueva versión, se encuentra funcional y conformado por importantes partes:

- El backend que se encuentra en la carpeta `backend/`: API en NestJS + Prisma (SQLite).
- El frontend que se encuentra en la carpeta `frontend/`: SPA en React + Vite + Tailwind.

Detalles Técnicos

Tecnologías Utilizadas

Lista de tecnologías utilizadas en el proyecto:

Backend

El lenguaje principal del backend es `TypeScript`, que se utiliza junto con NestJS para construir la API, sin embargo, se detalla a continuación cada una de las tecnologías empleadas:

- NestJS: framework principal de la API
- Prisma: ORM para interacción con la base de datos SQLite
- SQLite: base de datos utilizada por Prisma
- `@nestjs/jwt` y `bcryptjs`: autenticación (tokens y hash de contraseñas)
- `multer`: maneja la subida de imágenes (portadas de libros, avatares de usuarios)
- `cookie-parser`: permite la lectura de las cookies
- `dotenv`: carga de variables de entorno

Frontend

Al igual que el backend, el frontend tiene como lenguaje principal `TypeScript`, a continuación se detallan cada una de las tecnologías empleadas:

- React: librería principal de la UI
- React Router DOM: ruteo en la SPA
- Tailwind CSS: facilita la creación de estilos

- Axios: cliente HTTP para realizar solicitudes al backend
- Zustand: manejador de estado global (sesión del usuario)
- Zod: validación de formularios
- `lucide-react`: íconos
- `es-toolkit`: utilidades varias, muy similar a lodash pero con ventajas en rendimiento y tamaño del bundle
- Vite: motor principal para la construcción y desarrollo de la SPA

Endpoints y Rutas

De igual forma a continuación se detallan los endpoints del backend y las rutas del frontend:

Backend (endpoints)

Método	Ruta	Descripción
POST	<code>/auth/signup</code>	Crea una cuenta
POST	<code>/auth/login</code>	Inicia sesión
POST	<code>/auth/refresh</code>	Renueva el access token
POST	<code>/auth/logout</code>	Cierra sesión
GET	<code>/auth/me</code>	Perfil del usuario autenticado
PATCH	<code>/auth/me</code>	Actualiza el perfil
POST	<code>/auth/me/avatar</code>	Sube el avatar
GET	<code>/books</code>	Lista libros (público, oculta los no publicados sin sesión)
POST	<code>/books</code>	Crea un libro
GET	<code>/books/:id</code>	Detalle de un libro
PATCH	<code>/books/:id</code>	Edita un libro
PATCH	<code>/books/:id/status</code>	Publica o archiva un libro
POST	<code>/books/:id/cover</code>	Sube la portada
DELETE	<code>/books/:id</code>	Elimina el libro (y su historial de ventas/stock)

Método	Ruta	Descripción
GET	<code>/books/:id/stock</code>	Stock actual del libro
GET	<code>/books/:id/stock-movements</code>	Historial de movimientos de stock
GET	<code>/categories</code>	Lista categorías
POST	<code>/categories</code>	Crea una categoría
POST	<code>/stock-movements</code>	Registra una entrada o salida de stock
GET	<code>/sales</code>	Lista ventas
POST	<code>/sales</code>	Registra una venta
GET	<code>/sales/:id</code>	Detalle de una venta
PATCH	<code>/sales/:id</code>	Edita una venta
DELETE	<code>/sales/:id</code>	Elimina una venta
GET	<code>/metrics/overview</code>	Totales generales (ventas, ganancia, ROI)
GET	<code>/metrics/top-categories</code>	Categorías más vendidas
GET	<code>/metrics/top-cities</code>	Ciudades con más ventas

Frontend (rutas)

Ruta	Descripción
<code>/</code>	Landing pública
<code>/login</code>	Inicio de sesión
<code>/signup</code>	Registro de cuenta
<code>/dashboard</code>	Panel con métricas de ventas
<code>/books/published</code>	Vista interna de libros publicados
<code>/books/new</code>	Formulario para registrar un libro nuevo
<code>/books/:id/edit</code>	Edición y eliminación de un libro
<code>/books/manage</code>	Tabla de administración: publicar/archivar, editar, ajustar stock y eliminar

Ruta	Descripción
<code>/sales</code>	Historial de ventas, con registro y edición
<code>/profile</code>	Perfil del usuario (datos y avatar)

Requisitos

- Node.js 20 o superior
- npm 10 o superior

Instalación

Desde la raíz del repositorio, un solo comando instala las dependencias de ambos proyectos, ya que este repositorio fue configurado como un mono-repositorio:

```
npm install
```

Es básicamente el único comando necesario para instalar todas las dependencias de ambos proyectos.

Configuración

Cada proyecto tiene su propio archivo de variables de entorno, es cuestión de copiar los ejemplos (no es necesario ajustarlos):

```
cp backend/.env.example backend/.env
cp frontend/.env.example frontend/.env
```

backend/.env

Variable	Descripción
<code>PORT</code>	Puerto donde corre la API (<code>3000</code> por defecto)
<code>DATABASE_URL</code>	Ruta del archivo SQLite (por defecto <code>file:./dev.db</code>)
<code>JWT_SECRET</code>	String secreto para firmar los tokens de sesión
<code>FRONTEND_ORIGIN</code>	URL del frontend, usada para CORS (<code>http://localhost:5173</code> por defecto)

frontend/.env

Variable	Descripción
VITE_API_URL	URL del backend (http://localhost:3000 por defecto)

Con los valores de ejemplo funciona correctamente (localmente), no es obligatorio cambiarlos.

Base de datos

El backend usa [Prisma](#) como ORM y actualmente [SQLite](#) como base de datos (para no complicar la configuración local), sin embargo, se puede cambiar a otra base de datos soportada por Prisma si se desea. Antes de levantarlo por primera vez es necesario generar los archivos de cliente de Prisma y aplicar las migraciones, sobre el directorio raíz corre los siguientes comandos:

```
npm run generate
npm run migrate
```

Los comandos previos generan el archivo `backend/dev.db` vacío. Para obtener datos de ejemplo (categorías, libros, ventas) corre:

```
npm run seed
```

Ejecución

Para levantar frontend y backend juntos en modo desarrollo corre:

```
npm run all
```

- Backend: <http://localhost:3000> (puerto fijo, configurable con `PORT` en `backend/.env`)
- Frontend: <http://localhost:5173> (puerto fijo en `frontend/vite.config.ts`)

También se pueden levantar por separado desde el directorio raíz:

```
npm run backend # solo la API
npm run frontend # solo la SPA
```

Build de producción

```
npm run build
```

Compila ambos proyectos y como resultado se generan los directorios de salida `backend/dist` y `frontend/dist`.